



LISTA DE EXERCÍCIOS - SISTEMAS INTELIGENTES APLICADOS

Lista de Exercícios 5

Árvore de Decisão II

DOCENTE: Thiago França Naves

DATA: 30/09/2026

ALUNO: Mayara Rodrigues Pereira e Leonardo José Reis Pinto

As medidas das questões 1 a 3 foram calculadas com o script `solucao.py` (Apêndice A). Usamos as definições dos slides: entropia $H = -\sum p_i \log_2 p_i$, ganho de informação $G = H(\text{pai}) - \sum \frac{n_v}{n} H(v)$, índice Gini $1 - \sum p_i^2$ (ponderado pelos filhos, e o menor é o melhor) e razão de ganho $GR = G/\text{SplitInfo}$, em que $\text{SplitInfo} = -\sum \frac{n_v}{n} \log_2 \frac{n_v}{n}$ é a entropia da própria divisão.

Questão 1

José quer saber qual melhor estratégia para uma prova de SIA. [...] Ele tem 9 entradas da base dele e dois atributos: “ficar acordado até tarde antes da prova” (S) e “Se frequenta todas as aulas do dia” (A). Base(completa) = [5+, 4–]; Base(S+) = [3+, 2–], Base(S–) = [2+, 2–]; Base(A+) = [5+, 1–], Base(A–) = [0+, 3–]. a) Monte a árvore de decisão utilizando entropia e depois uma nova árvore usando Gini Index.

Resposta. Com entropia. A base tem $H = -\frac{5}{9} \log_2 \frac{5}{9} - \frac{4}{9} \log_2 \frac{4}{9} = 0,991$.

$$G(S) = 0,991 - \frac{5}{9} \cdot 0,971 - \frac{4}{9} \cdot 1 = 0,007$$

$$G(A) = 0,991 - \frac{6}{9} \cdot 0,650 - \frac{3}{9} \cdot 0 = 0,558$$

Frequentar as aulas (A) tem ganho muito maior e vai para a raiz.

Com Gini. A base tem $1 - (5/9)^2 - (4/9)^2 = 0,494$.

$$\text{Gini}(S) = \frac{5}{9} \cdot 0,480 + \frac{4}{9} \cdot 0,500 = 0,489$$

$$\text{Gini}(A) = \frac{6}{9} \cdot 0,278 + \frac{3}{9} \cdot 0 = 0,185$$

O menor Gini também é o de A, então a raiz é a mesma.

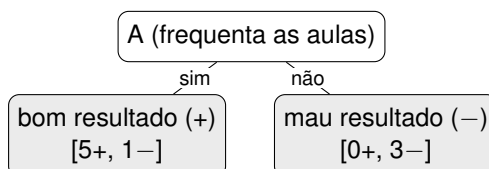


Figura 1. Árvore obtida tanto pela entropia quanto pelo Gini.

As duas medidas deram a mesma árvore. O ramo A– é puro. O ramo A+ ainda tem um exemplo negativo, mas não dá para dividi-lo por S, porque a lista só informa as contagens de S e A separadas, e não quantos dos 6 exemplos de A+ ficaram acordados até tarde. Então ele vira uma folha com a classe da maioria. Ficar acordado até tarde quase não muda a proporção de acertos (3/5 contra 2/4), por isso o ganho de S é praticamente zero. A conclusão para o José: o que importa é ir às aulas.

Questão 2

Você está preso em uma ilha deserta. Essa possui Cogumelos de vários tipos [...] Você tem os seguintes dados para considerar (exemplos A a H de treino e U, V e W). a) Monte a árvore de decisão utilizando Razão de Ganho.

Resposta. Usamos os exemplos A a H para montar a árvore, e U, V e W, que estão separados na tabela, para testar. No treino há 3 comestíveis (A, B, C) e 5 venenosos, com entropia 0,954.

Na raiz, NotHeavy, Smelly e Spotted dividem a base em 5 contra 3 e têm todos o mesmo ganho (0,003) e a mesma razão (0,0034). Smooth divide em 4 contra 4, com ganho 0,049, SplitInfo 1 e razão **0,049**, a maior, e vira a raiz.

No ramo Smooth = 1 (C, D, F, G, só C é comestível), Smelly separa tudo: C é o único que cheira. Ganho 0,811, SplitInfo 0,811, razão **1,0**. NotHeavy teve razão 0,311 e Spotted 0,151.

No ramo Smooth = 0 (A, B, E, H, com A e B comestíveis), Smelly também separa tudo: E e H cheiram e são venenosos. Razão **1,0**, contra 0,384 de NotHeavy e 0 de Spotted.

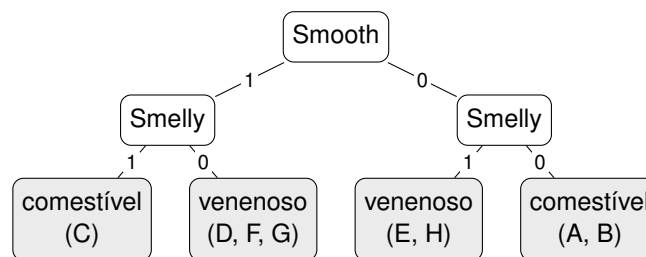


Figura 2. Árvore montada pela razão de ganho.

A árvore classifica o treino sem erro. No teste, U (liso e com cheiro) sai comestível e acerta. V também é liso e com cheiro e sai comestível, mas é venenoso. W não é liso, tem cheiro, sai venenoso e é comestível. São 1 acerto em 3. U e V caem na mesma folha e têm classes opostas, então nenhuma árvore que use só Smooth e Smelly acertaria os dois. Com 8 exemplos, a regra “liso com cheiro é comestível” se apoia num único cogumelo (C), e comer V com base nela seria um risco. A árvore é boa para explicar o treino, mas a base é pequena demais para confiar nela.

Questão 3

Em setembro de 2012, 800 planetas extra-solares foram identificados [...] você descobrirá árvores de decisão para prever se um planeta é habitável com base apenas em recursos observáveis usando telescópios. a) Monte as árvores de decisão utilizando ganho de informação, gini index e razão de ganho.

Resposta. Cada linha da tabela representa vários planetas (coluna Count), então as contas usam as contagens como pesos. Dos 800 planetas, 374 são habitáveis e 426 não (entropia 0,997, Gini 0,498).

Tabela 1. Medidas na raiz.

Atributo	Valor 1 (Yes/No)	Valor 2 (Yes/No)	Ganho	Gini	Razão
Size	Big 190/160	Small 184/266	0,0128	0,4891	0,0130
Orbit	Near 159/141	Far 215/285	0,0068	0,4932	0,0071

Os três critérios escolhem **Size** para a raiz: maior ganho, menor Gini e maior razão de ganho. Os

SplitInfo dos dois atributos são parecidos (0,989 e 0,954), então a razão não muda a ordem do ganho. Depois disso só sobra Orbit, que é usado nos dois ramos:

Tabela 2. Segundo nível, com Orbit dentro de cada tamanho.

Ramo	Near (Yes/No)	Far (Yes/No)	Ganho	Gini	Razão
Size = Big	20/130	170/30	0,403	0,245	0,410
Size = Small	139/11	45/255	0,443	0,215	0,483

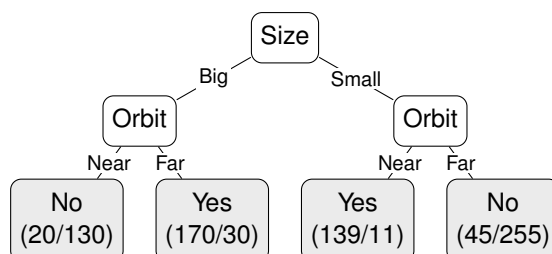


Figura 3. Árvore para os três critérios. Nas folhas, habitáveis/não habitáveis.

As três árvores ficaram iguais e erram 106 dos 800 planetas (86,75% de acerto). Cada atributo sozinho quase não ajuda: só Size acerta 57% e só Orbit 55,5%, perto dos 53% de dizer sempre “não”. Os dois juntos são bons porque a relação é cruzada: planeta grande é habitável se estiver longe, e pequeno se estiver perto. Por isso os ganhos na raiz são tão baixos (0,013) e os do segundo nível tão altos (acima de 0,4). Se Orbit fosse para a raiz, o resultado final seria o mesmo, com os mesmos 106 erros.

Questão 4

O que é e como funciona a estratégia de construção de árvores de decisão por indução? Porque indução não garante a melhor árvore possível em um determinado problema/base de dados.

Resposta. Indução é generalizar a partir de exemplos: olhando casos particulares rotulados, o algoritmo tira uma regra geral, que é a árvore. A construção é de cima para baixo e por divisão. Começa com todos os exemplos na raiz, escolhe o atributo que melhor separa as classes por uma medida de impureza (ganho de informação, Gini ou razão de ganho), divide os exemplos pelos valores desse atributo e repete o processo em cada subconjunto. Para quando o nó fica puro, os atributos acabam ou há poucos exemplos, e a folha recebe a classe da maioria.

Isso não garante a melhor árvore por alguns motivos:

- A escolha é gulosa. Em cada nó o algoritmo pega o atributo que parece melhor naquele momento e nunca revê a decisão. Um atributo fraco na raiz pode levar a uma árvore menor depois, como nas questões 3 da lista 4 e 3 desta lista, em que os atributos só são úteis juntos.
- Achar a menor árvore consistente com os dados é um problema NP-completo. Testar todas as árvores é inviável, então se aceita uma heurística.
- A árvore é induzida de uma amostra, e não da população. Com poucos exemplos, ruído ou exemplos contraditórios, ela descreve bem o treino e erra em casos novos, como os cogumelos da questão 2.
- Cada medida tem seu viés. O ganho de informação puro favorece atributos com muitos valores, e é por isso que existe a razão de ganho.

A heurística costuma dar árvores boas e pequenas, mas não há garantia de que a árvore obtida seja a melhor possível.

Apêndice A. Código

python solucao.py

```
# Lista 5 - Árvore de Decisão II
# Sistemas Inteligentes Aplicados - UTFPR
# Uso: python solucao.py

from math import log2

def entropia(contagens):
    total = sum(contagens)
    h = 0
    for n in contagens:
        if n:
            h -= n / total * log2(n / total)
    return h

def gini(contagens):
    total = sum(contagens)
    return 1 - sum((n / total) ** 2 for n in contagens)

def medidas(pai, filhos):
    # pai e filhos são listas de contagens por classe
    total = sum(pai)
    pesos = [sum(f) / total for f in filhos]
    ig = entropia(pai) - sum(p * entropia(f) for p, f in zip(pesos, filhos))
    g = sum(p * gini(f) for p, f in zip(pesos, filhos))
    split = entropia([sum(f) for f in filhos])
    gr = ig / split if split else 0
    return ig, g, gr, split

# questao 1
print("[1] José e a prova")
S = [5, 4]
filhos = {"S": [[3, 2], [2, 2]], "A": [[5, 1], [0, 3]]}
print(f" base: entropia={entropia(S):.4f} gini={gini(S):.4f}")
for a, f in filhos.items():
    ig, g, gr, _ = medidas(S, f)
    print(f" {a}: filhos {f} entropias {entropia(f[0]):.4f} "
          f"{entropia(f[1]):.4f} ganho={ig:.4f} "
          f"gini dos filhos {gini(f[0]):.4f} {gini(f[1]):.4f} "
          f" gini ponderado={g:.4f}")

# questao 2
print("\n[2] cogumelos")
nomes = ["NotHeavy", "Smelly", "Spotted", "Smooth"]
treino = {"A": ([1, 0, 0, 0], 1), "B": ([1, 0, 1, 0], 1),
          "C": ([0, 1, 0, 1], 1), "D": ([0, 0, 0, 1], 0),
          "E": ([1, 1, 1, 0], 0), "F": ([1, 0, 1, 1], 0),
          "G": ([1, 0, 0, 1], 0), "H": ([0, 1, 0, 0], 0)}
teste = {"U": ([0, 1, 1, 1], 1), "V": ([1, 1, 0, 1], 0),
         "W": ([1, 1, 0, 0], 1)}

def contar(ids):
    comest = sum(treino[i][1] for i in ids)
    return [comest, len(ids) - comest]

def montar(ids, livres, nivel=0):
    c = contar(ids)
    recuo = " " + " " * nivel
    if c[0] == 0 or c[1] == 0 or not livres:
        classe = 1 if c[0] >= c[1] else 0
        print(f"{recuo}folha {ids} -> {'comestivel' if classe else 'venenoso'}")
        return classe
    melhor = None
```

```

for a in livres:
    lado1 = [i for i in ids if treino[i][0][a] == 1]
    lado0 = [i for i in ids if treino[i][0][a] == 0]
    ig, _, gr, split = medidas(c, [contar(lado1), contar(lado0)])
    print(f"{recuo}{nomes[a]}: ganho={ig:.4f} split={split:.4f}"
          f" razão={gr:.4f}")
    if melhor is None or gr > melhor[0]:
        melhor = (gr, a, lado1, lado0)
    _, a, lado1, lado0 = melhor
print(f"{recuo}-> divide por {nomes[a]}")
resto = [x for x in livres if x != a]
print(f"{recuo}{nomes[a]} = 1:")
sim = montar(lado1, resto, nivel + 1)
print(f"{recuo}{nomes[a]} = 0:")
nao = montar(lado0, resto, nivel + 1)
return (a, sim, nao)

arvore = montar(list(treino), [0, 1, 2, 3])

def classificar(no, x):
    while isinstance(no, tuple):
        a, sim, nao = no
        no = sim if x[a] == 1 else nao
    return no

print(" teste:")
for nome, (x, real) in teste.items():
    prev = classificar(arvore, x)
    print(f" {nome}: real={real} previsto={prev}")

# questao 3
print("\n[3] planetas")
planetas = [("Big", "Near", "Yes", 20), ("Big", "Far", "Yes", 170),
             ("Small", "Near", "Yes", 139), ("Small", "Far", "Yes", 45),
             ("Big", "Near", "No", 130), ("Big", "Far", "No", 30),
             ("Small", "Near", "No", 11), ("Small", "Far", "No", 255)]

def cont(filtro):
    sim = sum(n for s, o, h, n in planetas if filtro(s, o) and h == "Yes")
    nao = sum(n for s, o, h, n in planetas if filtro(s, o) and h == "No")
    return [sim, nao]

raiz = cont(lambda s, o: True)
print(f" total: {raiz} entropia={entropia(raiz):.4f} gini={gini(raiz):.4f}")
attrs = {"Size": (0, ["Big", "Small"]), "Orbit": (1, ["Near", "Far"])}
for nome, (i, vals) in attrs.items():
    filhos = [cont(lambda s, o, v=v: (s, o)[i] == v) for v in vals]
    ig, g, gr, split = medidas(raiz, filhos)
    print(f" {nome}: {dict(zip(vals, filhos))} ganho={ig:.4f}"
          f" gini={g:.4f} split={split:.4f} razão={gr:.4f}")

print(" segundo nivel:")
for nome, (i, vals) in attrs.items():
    outro = [k for k in attrs if k != nome][0]
    j, vals2 = attrs[outro]
    for v in vals:
        pai = cont(lambda s, o: (s, o)[i] == v)
        filhos = [cont(lambda s, o, w=w: (s, o)[i] == v and (s, o)[j] == w)
                  for w in vals2]
        ig, g, gr, split = medidas(pai, filhos)
        folhas = ", ".join(f"{w}: {f} -> {'Yes' if f[0] > f[1] else 'No'}"
                           for w, f in zip(vals2, filhos))
        print(f" raiz {nome}={v} {pai}: {outro} ganho={ig:.4f}"
              f" gini={g:.4f} razão={gr:.4f} | {folhas}")

# erros de cada árvore no conjunto de 800 planetas
for nome, (i, vals) in attrs.items():
    outro = [k for k in attrs if k != nome][0]
    j, vals2 = attrs[outro]

```

```

erros = 0
for v in vals:
    for w in vals2:
        f = cont(lambda s, o: (s, o)[i] == v and (s, o)[j] == w)
        erros += min(f)
    print(f"  árvore com {nome} na raiz e {outro} embaixo: {erros} erros"
          f" em 800 ({100 * (800 - erros) / 800:.2f}% de acerto)")
for nome, (i, vals) in attrs.items():
    erros = sum(min(cont(lambda s, o, v=v: (s, o)[i] == v)) for v in vals)
    print(f"  só {nome}: {erros} erros ({100 * (800 - erros) / 800:.2f}%)")

```